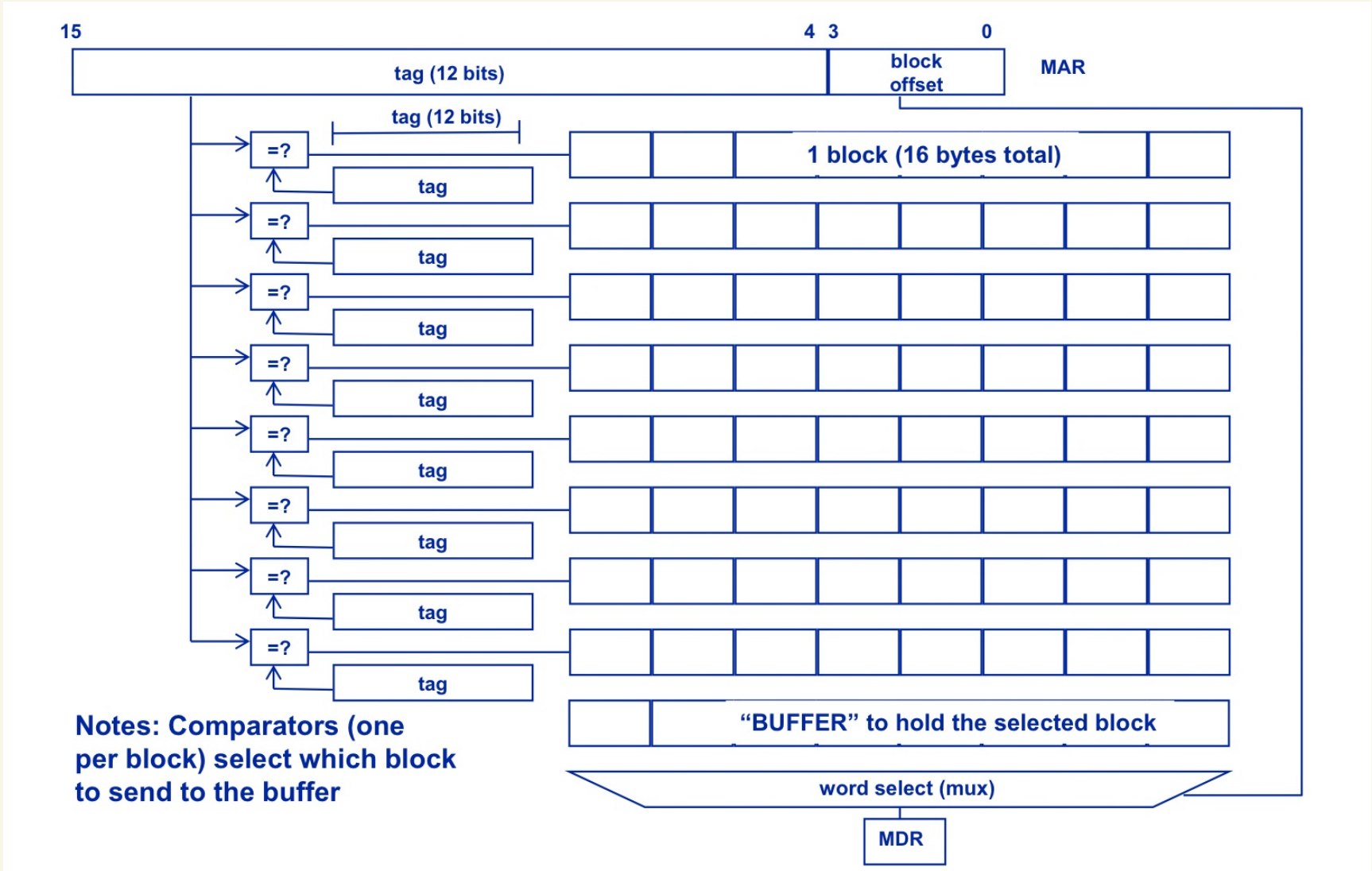
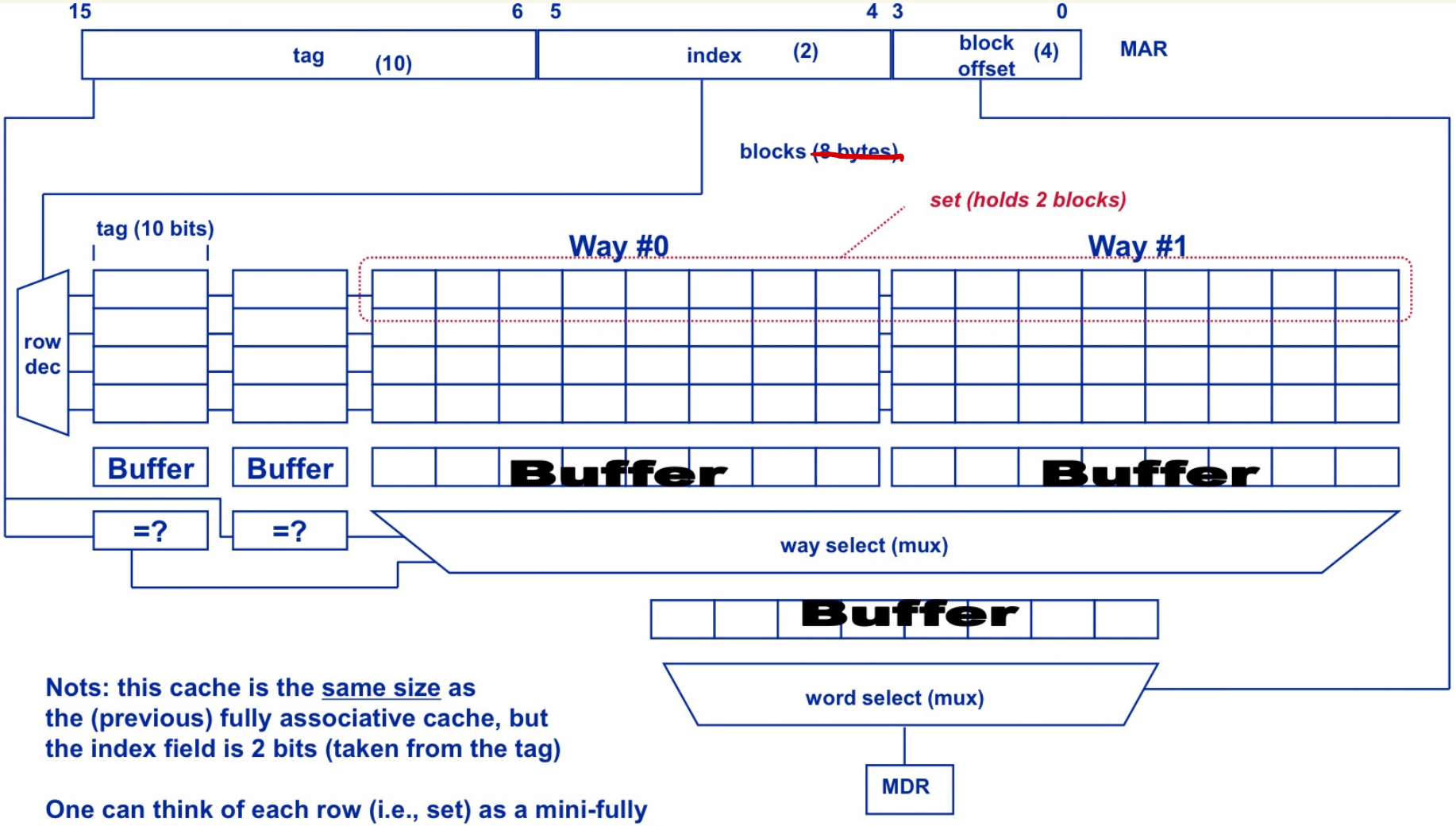
Fully Associative



2^7 bytes/$, 2^4 bytes/block, 2^3 blocks/set
(C=7, B=4, S=3)

2-way Set Associative

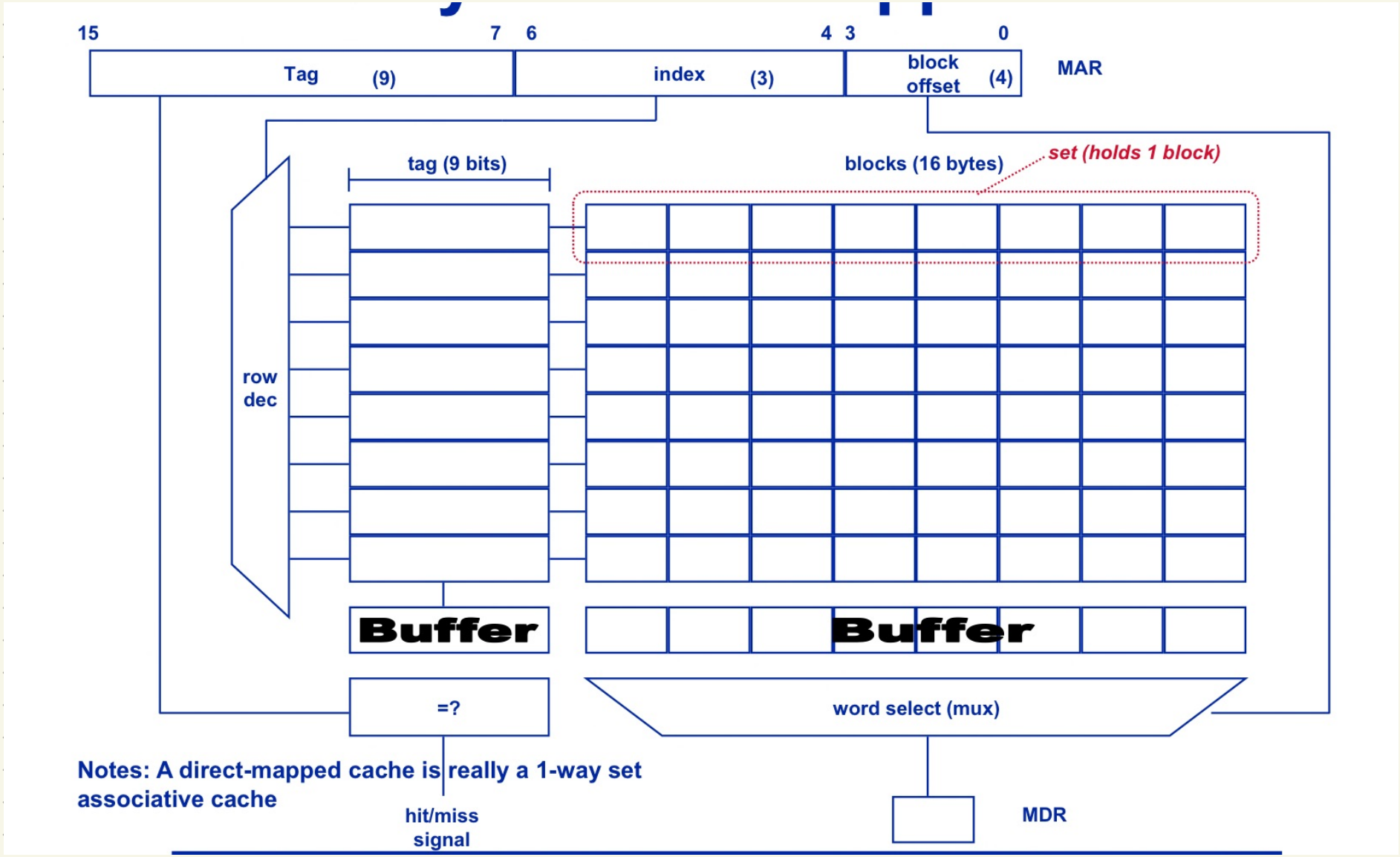


Nots: this cache is the same size as the (previous) fully associative cache, but the index field is 2 bits (taken from the tag)

One can think of each row (i.e., set) as a mini-fully associative cache. The index picks which mini-fully associative cache to search

$C = 7, \quad B = 4, \quad S = 1$

1-way or Direct Mapped cache ($)



(C = 7, B = 4, S = 0)

Cache Dimensions

2^C total # of bytes of data in $

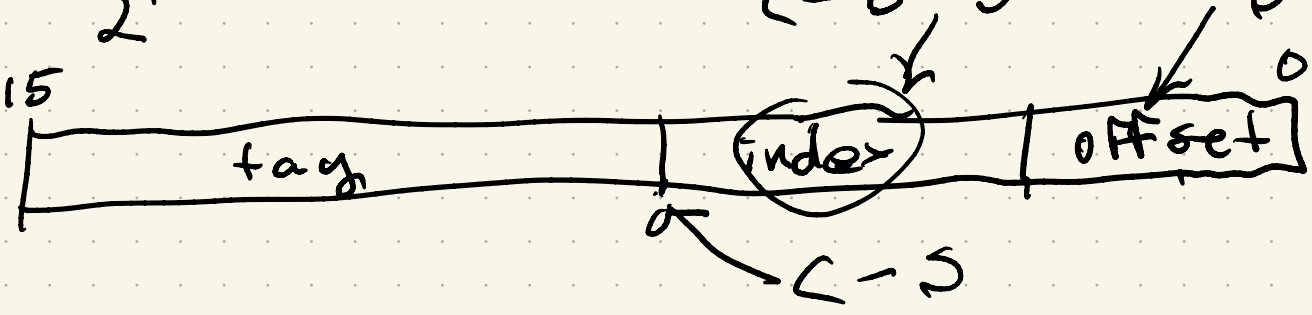
2^B total bytes per block

2^S total # blocks per each set

(C, B, S)

$S = 0$, Direct-mapped cache

$2^S = \frac{2^C}{2^B} \Rightarrow S = C - B$ Fully associative



Cache performance

1. Hit — Hit time
2. Hit time + Miss penalty

$$\text{Probability of miss} = \frac{\#\text{ misses}}{\text{total accesses}}$$

(Miss ratio)

Average Access Time (AAT)

= Hit time + (Miss rate) × (Miss penalty)

$$AAT = HT + MR \times MP$$

Cache management policies

Block placement -
bit selection

Block replacement -
if a set is full, replace block
to make room for new one

Write strategy

Block replacement

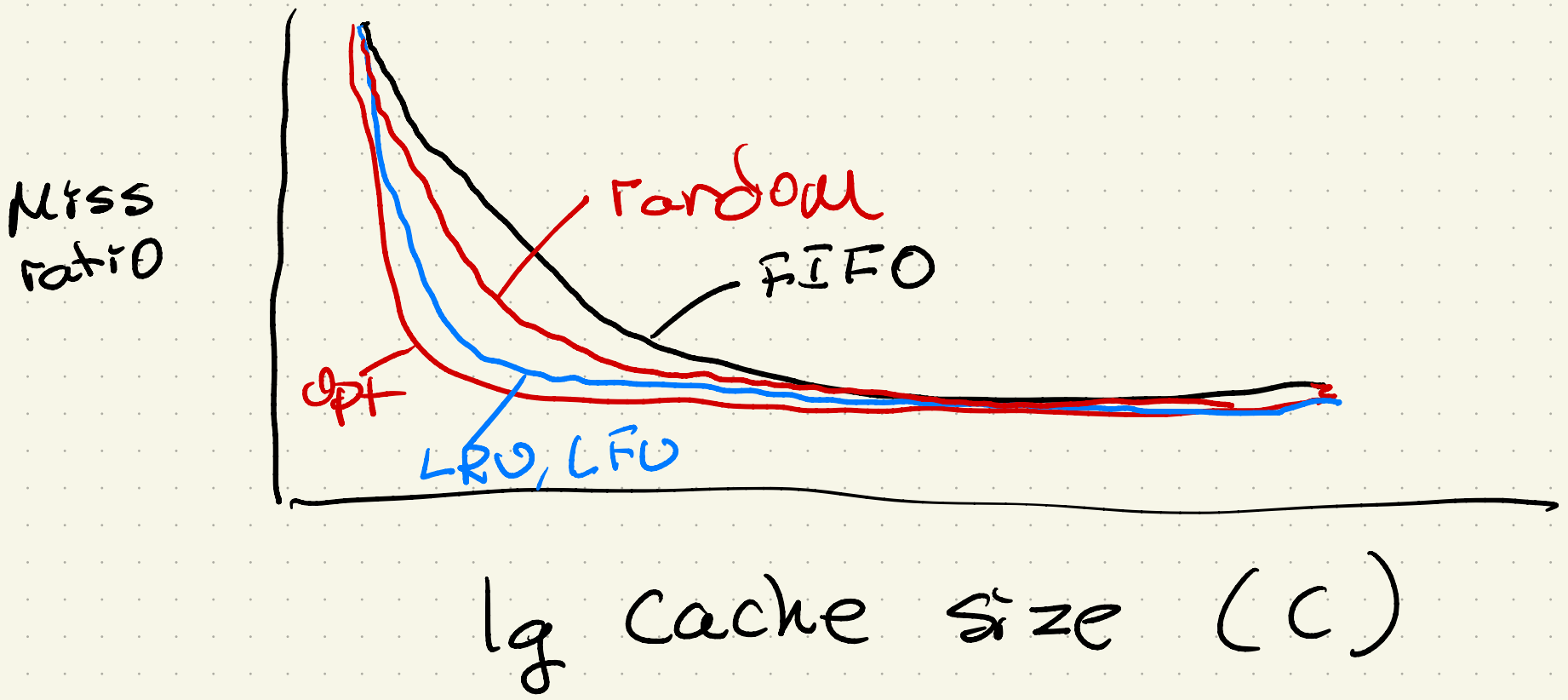
First in, first out (FIFO)
- replace oldest block

Optimal
- replace block used (if at all) farthest in the future

Least recently used (LRU)
- approximates Opt

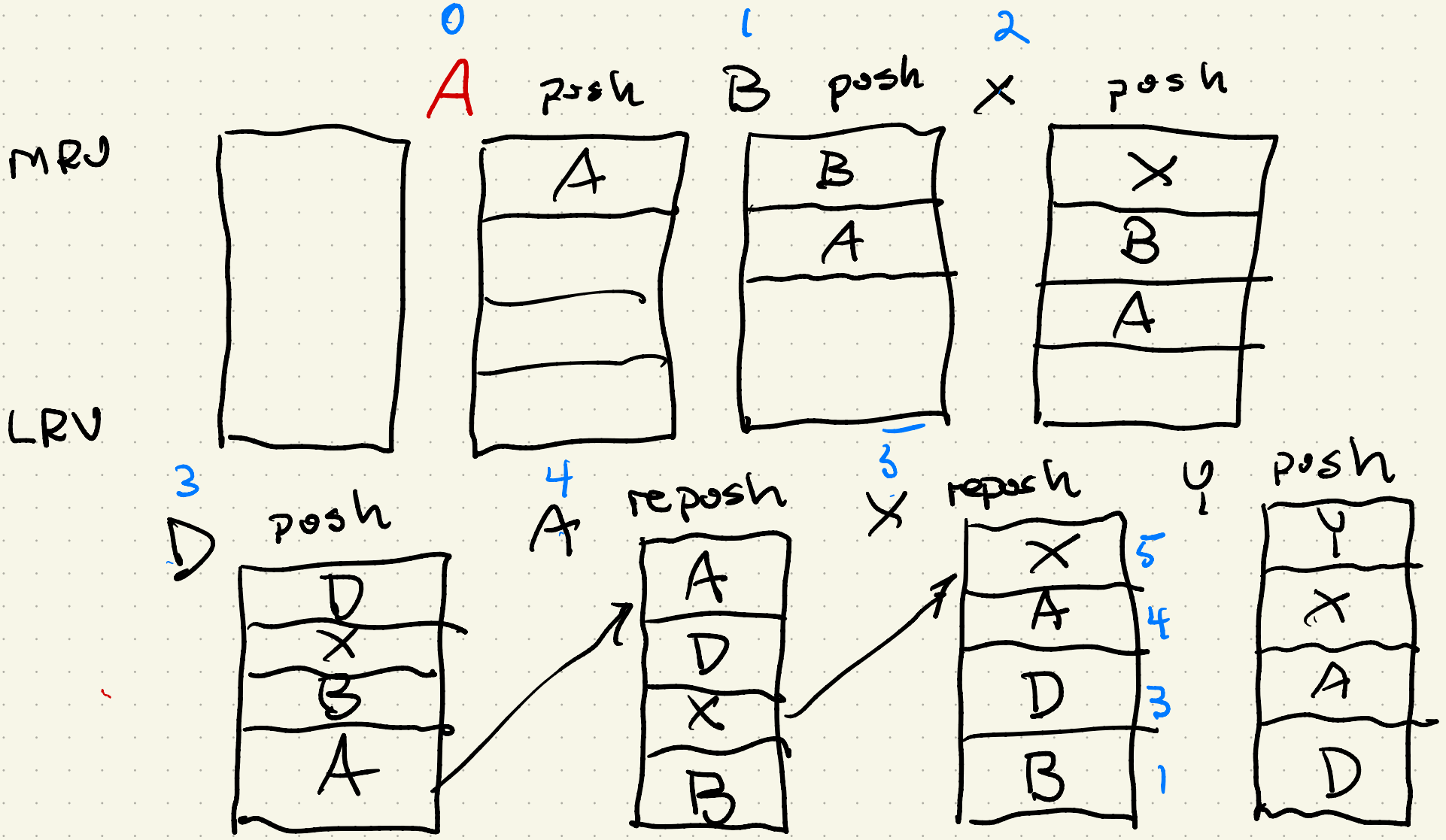
Least frequently used (LFU)
- Also close to Opt

Random


Miss ratio
random
FIFO
opt
LRU, LFU
lg cache size (C)

LRU is a "stacking algorithm"

Four block Fully associative cache



Using LRU stack for simulation

In stack?

No: push

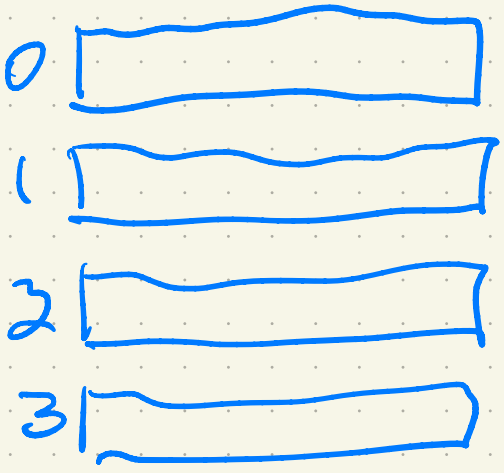
Yes: repush

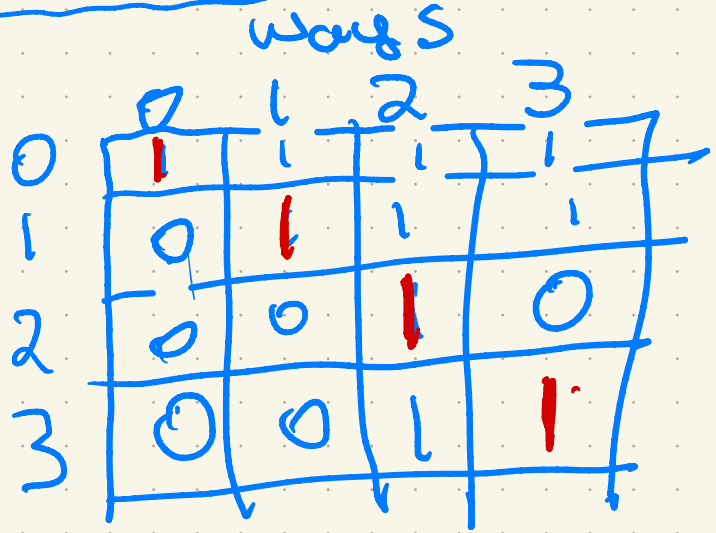
if found at depth d

Miss for all caches size < d

Hit for all caches size ≥ d

Aside





Access (Hit) on way i

set rows to 1

set cl to 0

0, 1, 2, 3, 1, 0, Miss